

Tiny object detection with SNIP, SNIPER and AutoFocus

by LAB Team



CONTENT

SNIP

Scale Normalization for
Image Pyramids

01

SNIPER

Scale Normalization for
Image Pyramids with
Efficient Resampling

02

AutoFocus

FocusPixels -
FocusChips -
FocusStacking

03



SNIP

Scale Normalization for Image Pyramids



Image classification at multiple scales

Naive Multi-scale Inference:

- CNN-B model
 - Step 1: downsample ImageNet dataset from 224 to 48, 64, 80, 96, 128
 - Step 2: upsample above images back to 224
 - Step 3: train CNN (ResNet101) that pretrained with 224 images

=> The more different in resolution between train set and test set, the lower accuracy

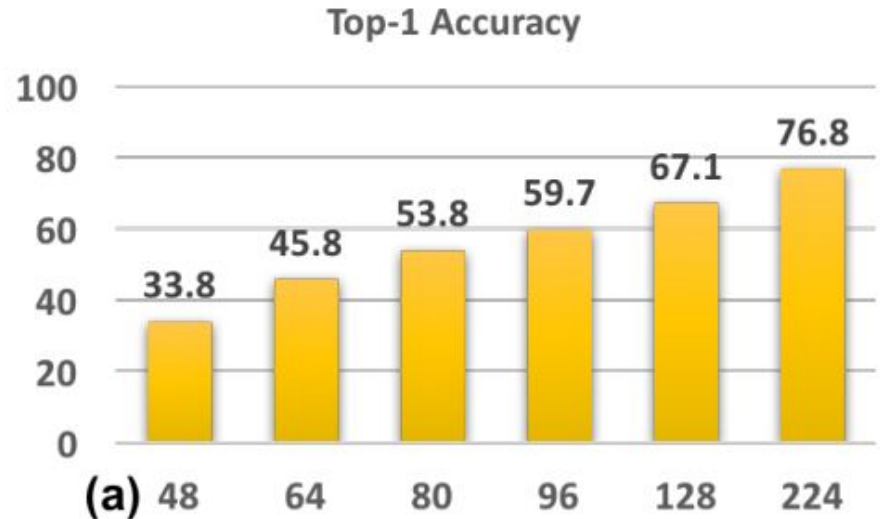


Image classification at multiple scales

Resolution Specific Classifiers:

- CNN-B: First conv layer is 7x7, stride = 2, follow by max pooling stride = 2x2
- CNN-S:
 - First conv layer is 3x3, stride = 1 (for 48x48 images)
 - First conv layer is 5x5, stride = 2 (for 96x96 images)
 - Data augmentation: random crop, color augmentation (in first 70 epochs)

Image classification at multiple scales

Fine-tuning High-resolution Classifiers:

- CNN-B-FT:
 - Fine-tuning CNN-B on up-sampled low resolution images

Image classification at multiple scales

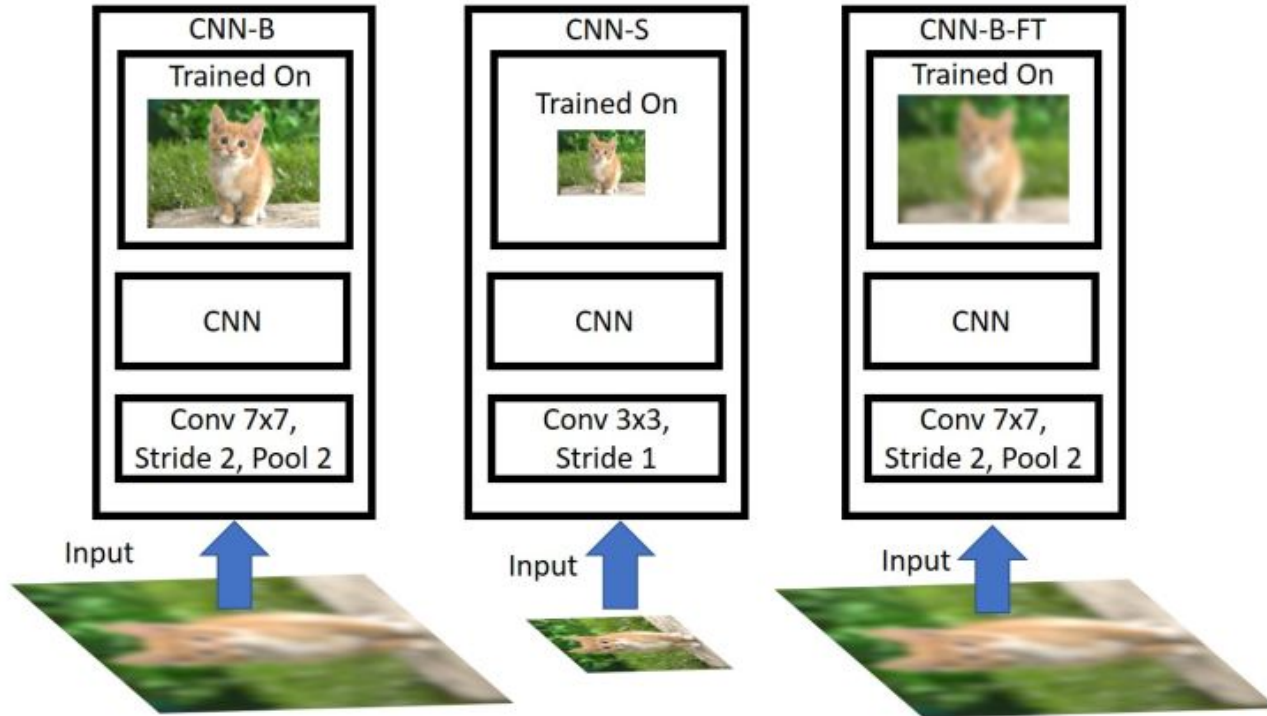
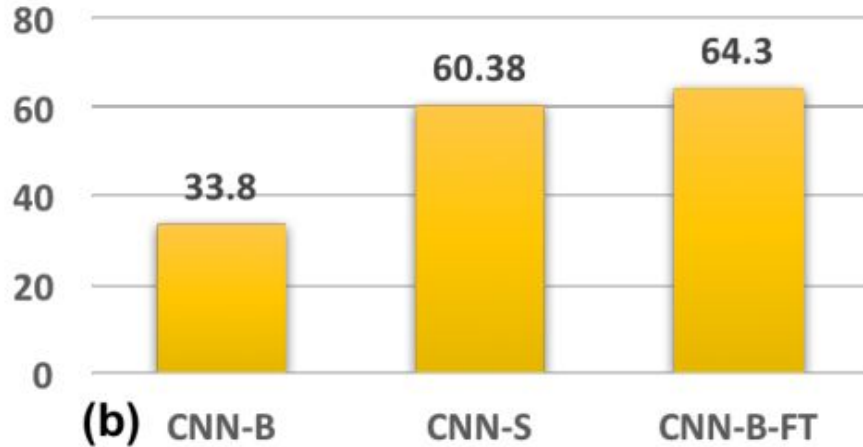
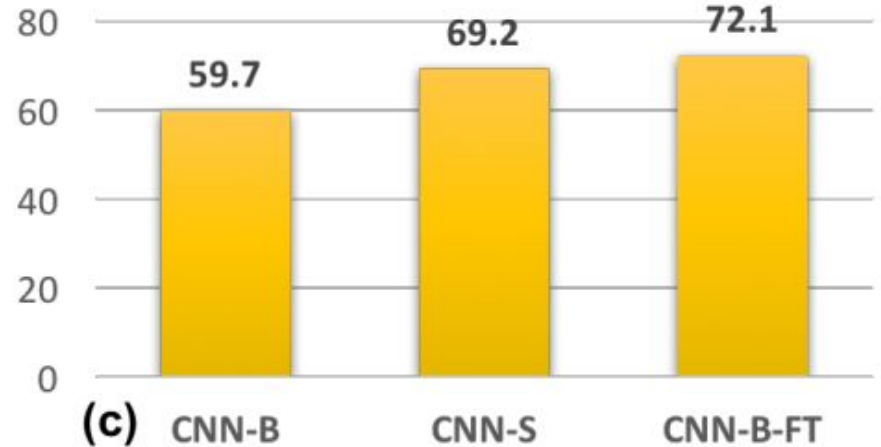


Image classification at multiple scales

48x48 Top-1 Accuracy



96x96 Top-1 Accuracy



Data Variation vs Correct Scale

- Train with different settings
- Evaluate on 1400x2000 images in COCO dataset (small object is smaller than 32x32 pixels)

Data Variation vs Correct Scale

Train with different resolution:

- 800_{all} : Train with image 800x1400, backprop all object instances
- 1400_{all} : Train with 1400x2000, backprop all object instances

=> $1400_{all} > 800_{all}$ (19.9 > 19.6 in mAP)

=> Improvement is not significant

=> with 1400_{all} , scaling up small objects improves, but medium-to-large objects become too large and degrade model performance

Data Variation vs Correct Scale

Scale specific detector:

- $1400_{<80px}$: Train with 1400x2000, backprop only object instances that smaller than 80x80 pixels

=> $800_{all} > 1400_{<80px}$ (19.6 > 16.4 in mAP)

=> Significantly worse

=> with $1400_{<80px}$, lots of data of medium-to-large instance have been ignored (30% of the total object instances)

Data Variation vs Correct Scale

Multi scale training:

- MST: Train with images that randomly sample from multiple resolutions

=> $800_{\text{all}} \sim \text{MST} (19.6 > 19.4 \text{ in mAP})$

=> Almost similar

=> with MST, instances are observed in multiple resolutions, but model cannot learn extreme small or large objects

Data Variation vs Correct Scale

Scale Normalization for Image Pyramids:

- SNIP:
 - Modified version of MST
 - Backprop only instances whose resolution is close to pretrained dataset (or fall in the desired scale range)
 - Ignore the remainder

=> Reduce the domain-shift and work in detecting small objects

$1400_{<80px}$	800_{all}	1400_{all}	MST	SNIP
16.4	19.6	19.9	19.5	21.4

SNIPER

Scale Normalization for Image Pyramids with Efficient Resampling



Problem of SNIP

- Process image multiple times
- Process all pixels of image

=> **SLOW**

=> **SNIPER** performs as **well** as **SNIP** while **reduce** number of **pixels** processed by a **factor of 3** on **COCO** dataset

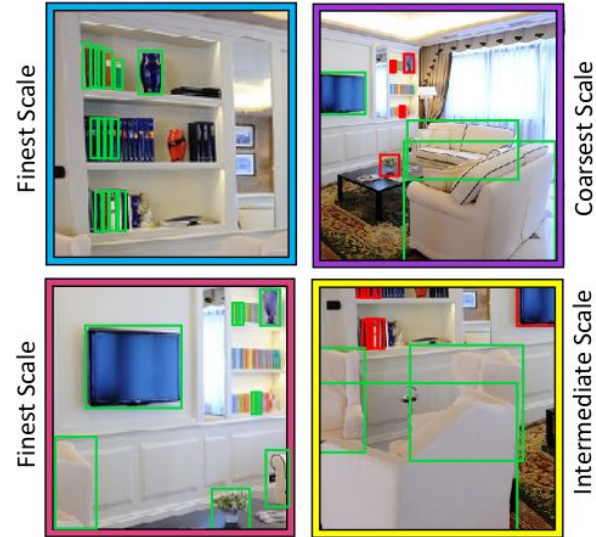
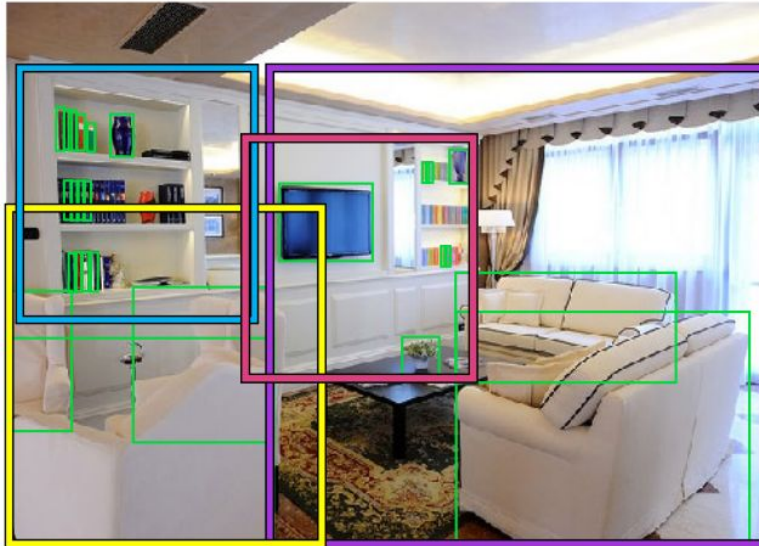
Chip Generation

- Generate chips C^i at scale s_i
 - Resize image
 - Place $K \times K$ pixel chip at equal intervals of d pixels
- => List of chips**



Positive Chip Selection

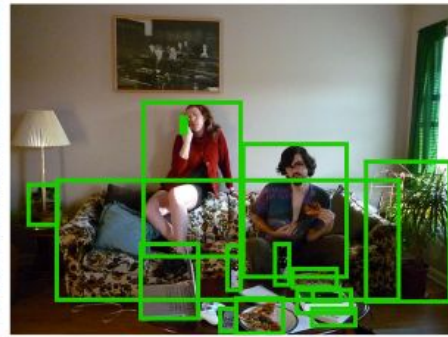
- Propose desired area for each scale $R^i = [r_{\min}^i, r_{\max}^i]$
- => List of valid bbox for each scale G_i
- Select chip that completely cover a bbox
- => List of positive chips C_{pos}^i



Negative Chip Selection

- Train RPN for a couple of epochs (no negative chips used)
 - Generate proposals over the entire training set
 - Assume: no proposals generated in portion = no object in portion
 - Remove all the proposals covered in C_{pos}^i
 - Select chip that cover at least M proposals in R^i
- => List of negative chips C_{neg}^i

Positive and Negative Chips



AUTO FOCUS

FocusPixels - FocusChips - Focus Stacking



Problem of SNIPER

- Process the entire high resolution image during inference

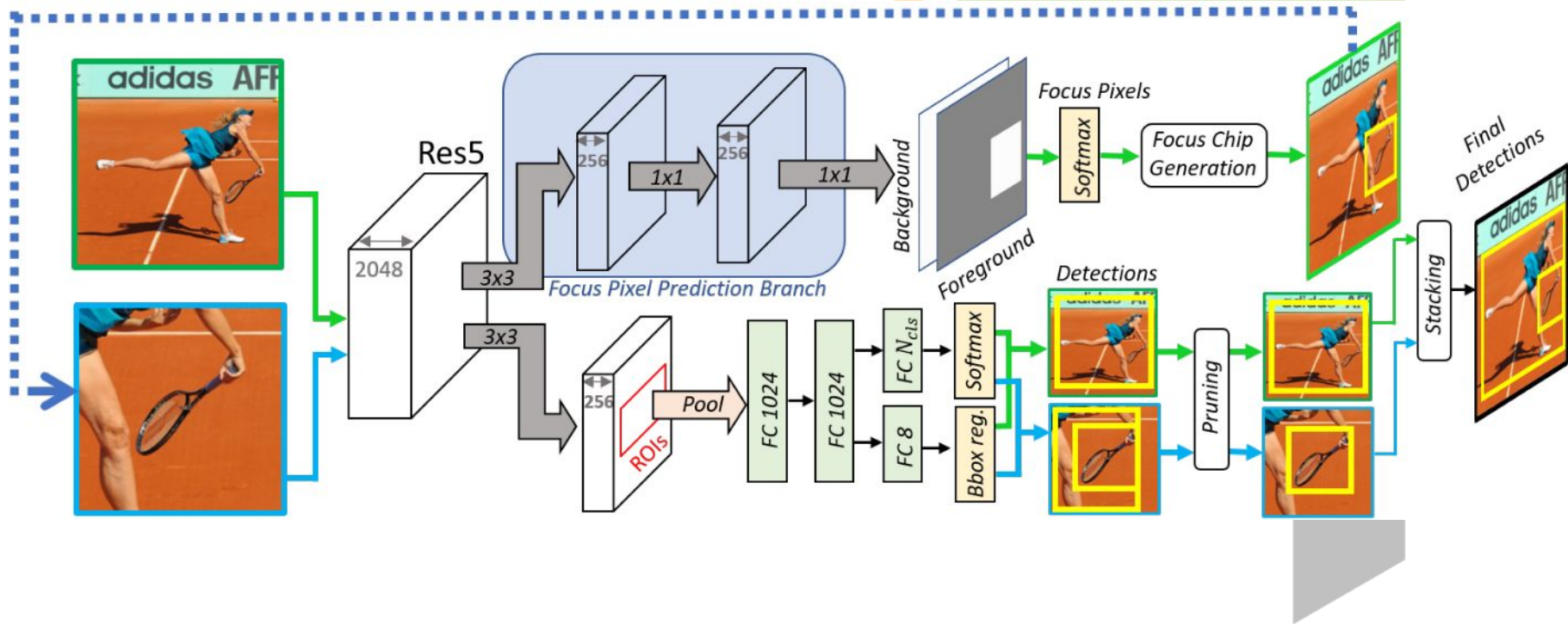
=> **SLOW**

=> **AutoFocus** proposes a framework that can **automatically predict** these chips for **small objects** at a **coarser scale**

Moreover,

AutoFocus proposes an algorithm which correctly **merges detections** from chips at **multiple scales**

AutoFocus



FocusPixels

- Author **labels pixel** from **features map Res5** of ResNet101
- A **pixel** is a **FocusPixel** if it has any **overlap** with a **small object** (range between **5x5** and **64x64**) => positives
- A **pixel** is invalid if it has **overlap** with **object** (range smaller than 5x5 or between **64x64** and **90x90**)
- All other pixels are negatives

$$l = \begin{cases} 1, & IoU(GT, l) > 0, a < \sqrt{GTArea} < b \\ -1, & IoU(GT, l) > 0, \sqrt{GTArea} < a \\ -1, & IoU(GT, l) > 0, b < \sqrt{GTArea} < c \\ 0, & \text{otherwise} \end{cases}$$

FocusChips

Algorithm 1: FocusChip Generator

Input : Predictions for feature map $\mathcal{P}$, threshold t ,
dilation constant d , minimum size of chip k

Output: Chips $\mathcal{C}$

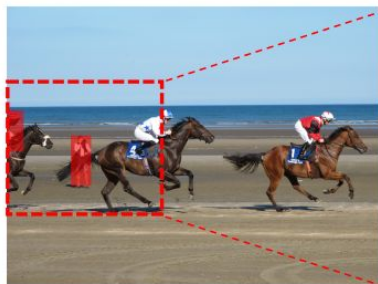
- 1 Transform $\mathcal{P}$ into a binary map using the threshold t
 - 2 Dilate $\mathcal{P}$ with a $d \times d$ filter
 - 3 Obtain a set of connected components $\mathcal{S}$ from $\mathcal{P}$
 - 4 Generate enclosing chips $\mathcal{C}$ of size $> k$ for each component in $\mathcal{S}$
 - 5 Merge chips $\mathcal{C}$ if they overlap
 - 6 **return** Chips $\mathcal{C}$
-

FocusStacking

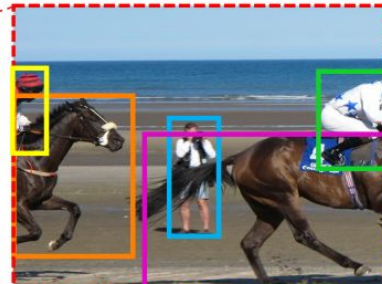
- **Cascaded multi-scale inference** causes generating some **false positives**
- Step 2 in FocusChips Generator ensures **no *interesting* object** at the next scale is at the **boundaries** of the **chip**
- These ***interesting* object** should be in **another chip**
- If **detections** are observed at the **boundaries**, **discard** them
 - Detail of discarding policy is based on detection boundary, chip boundary and image boundary
- **Non-Maximum Suppression** is applied to aggregate the detections



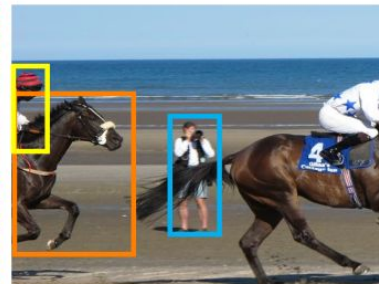
(a)



(b)



(c)



(d)

